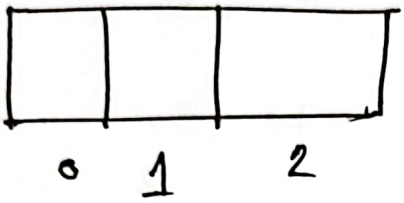
07-08-23

FORWARD CHAINING

given_arr = ["AB", "ZY", "MN", "OP"]

ht =



H = (Σ of alphabetic serial) % 3

H(AB) = (0+1) % 3 = 1

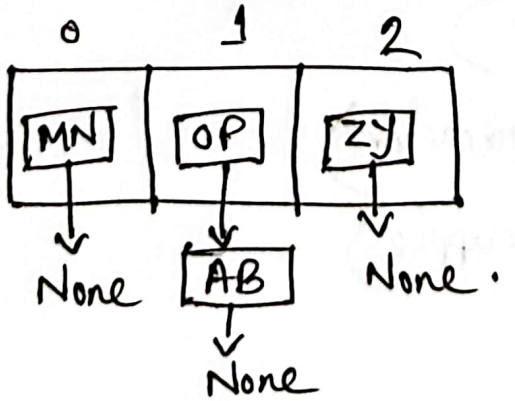
→ Alphabetic order.

H(ZY) = (25+24) % 3 = 29 % 3 = 2

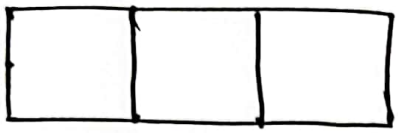
H(MN) = 0

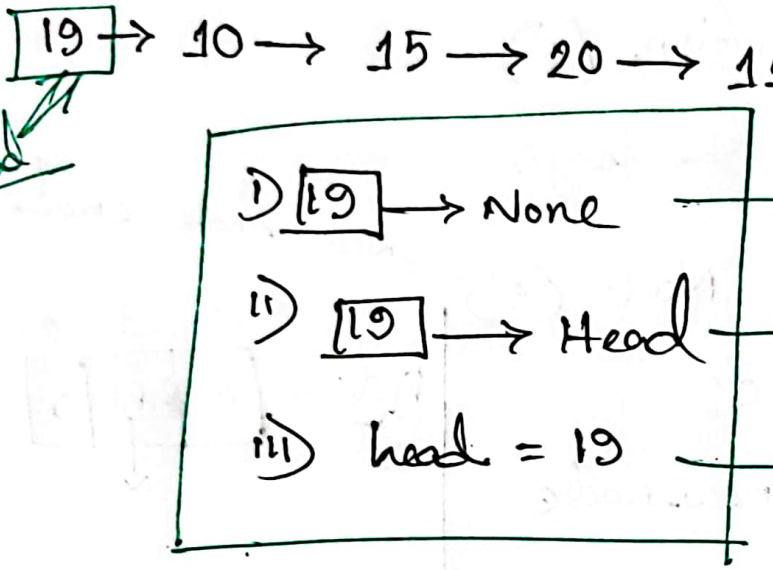
H(OP) = 1.

ht =

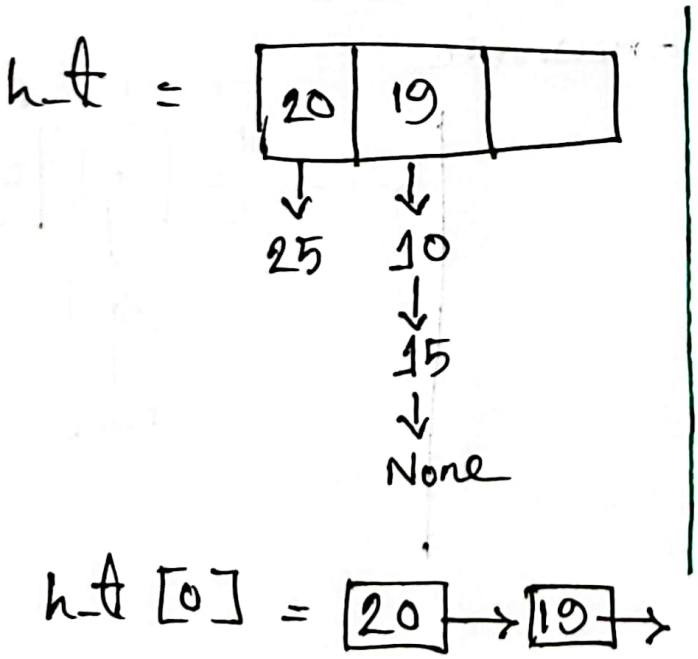


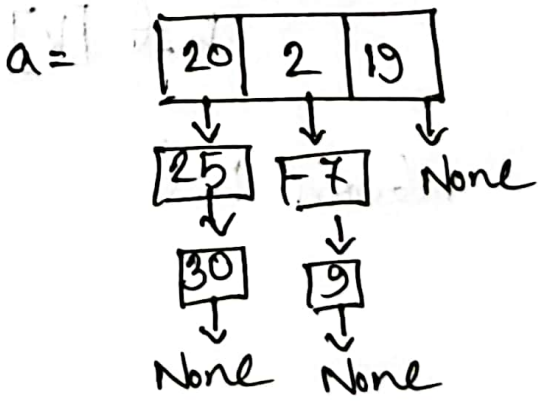
ht =





an array of linked list





temp = head

temp = a[1].

Healthcare

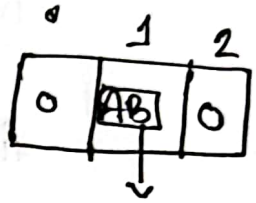
Emistat®
Ondansetron

CamScanner

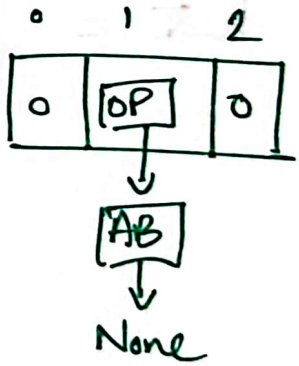
Hash table

```
h_t = np.zeroes (3)
for e in given_array:
    K = hash_function (e)
    ↳ generating the key.
    new_node = Node (e)  → New node create
    if h_t[k] == 0:
        h_t [k] = new_node
    else:
        new_node.next = h_t[k]
        h_t [k] = newnode
return h_t
```

h_t =



h_t =



4) key Indexing:

→ size of aux_arr = 9

given_arr = [2, 5, 8, 7, 3]

① you create an auxiliary array.

[size = max element of given_arr + 1]

aux_arr =

0	1	2	3	4	5	6	7	8
0	0	1	1	0	1	0	1	1

updated

does 8 exist in given_arr?

↳ print (aux_arr[8] == 1)

↳ whether it's true / false.

② aux_arr → 1 loop.

given_arr = [2, 5, 8, 7, 2, 3, 5]

aux_arr =

0	1	2	3	4	5	6	7	8
0	0	2	1	0	2	0	1	1

↳ 2 2 3 5 5 7 8 → sorting by key indexing

Healthcare
Emistat®
Ondansetron
CamScanner

④ offset every element by the minimum number.

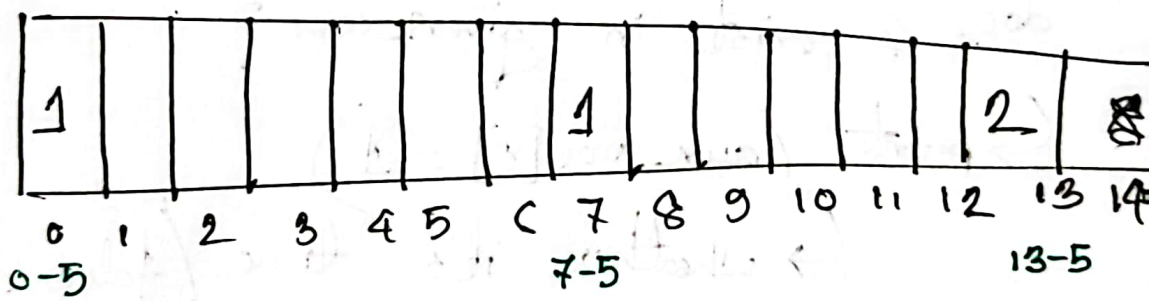
given_arr = [2, -5, 8, 7]

min = -5

given_arr = [2-(-5), (-5)-(-5), 8-(-5), 7-(-5)]

= [7, 0, 13, 12]

aux_arr =



⑤ does 13 exist in given_arr?

13-(-5) = 18.

→ -5 2 7 8

Sorting.

Hashing

- Time → saved
- Memory → over-used

Trade off between Time & Memory